Cryptanalysis and Performance Evaluation of Encryption Algorithms

May 7, 2026

Abstract

This project compares the security and computational performance of two encryption methods: the Caesar cipher and a custom polyalphabetic cipher ("Labyrinthe"). A brute-force attack is implemented in Python to evaluate their resistance and execution time.

1 Introduction

Cryptography is the science of securing information by transforming readable data into an unreadable form. The strength of an encryption system depends on the computational difficulty required to recover the original message without the key.

This study analyzes:

- Caesar Cipher
- Labyrinthe Cipher

2 Methods

2.1 Caesar Cipher

The Caesar cipher is a classical substitution method where each character is shifted by a fixed value.

Although historically important, it is insecure due to its very small key space.

2.2 Labyrinthe Cipher

The Labyrinthe cipher is a polyalphabetic encryption method that uses a repeating secret key to vary character shifts.

This increases complexity compared to the Caesar cipher.

3 Attack Method

We use a brute-force attack, which consists of:

- Testing all possible keys
- Decrypting the message for each key
- Comparing the result to the original message

This method is known as a **brute-force cryptanalysis attack**.

4 Results

Method	Time (s)	Security Level
Caesar Cipher	0.00003	Very weak
Labyrinthe Cipher	2.06	Stronger but not secure

Table 1: Comparison of execution time and security

5 Performance Visualization

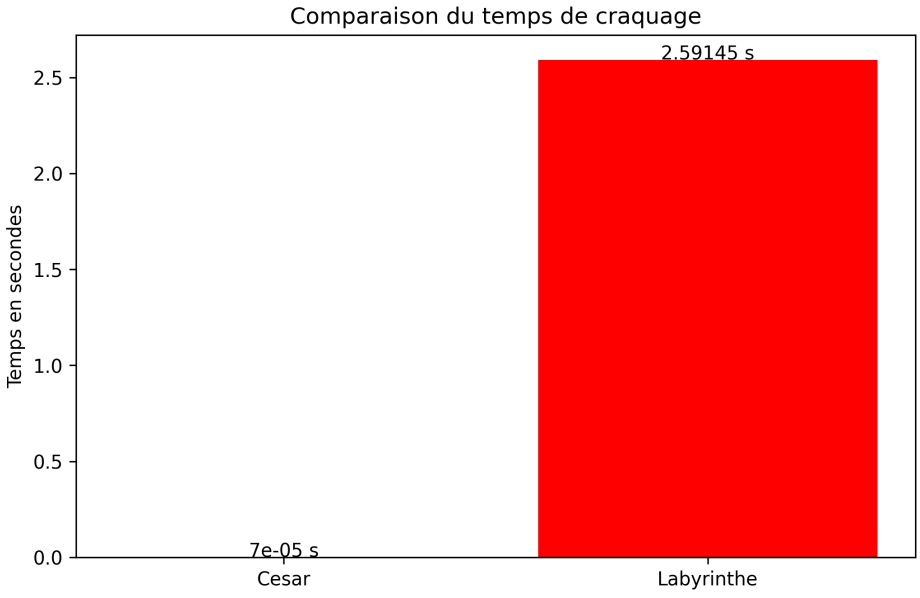

Figure 1: Execution time comparison between Caesar and Labyrinthe ciphers

The previous graph illustrates the execution time difference between the two encryption methods.

6 Complexity Analysis

The computational complexity of each method is:

$$\text{Caesar Cipher: } O(n)$$

$$\text{Labyrinthe Cipher: } O(n \cdot 26^k)$$

Where:

- n = message length
- k = key length

This shows that security increases exponentially with key size.

7 Code Summary

The Python implementation includes:

- Random message generation
- Caesar encryption and decryption
- Labyrinthe encryption
- Brute-force attack simulation
- Execution time measurement
- Graph visualization using Matplotlib

8 Key Definitions

Brute-force attack: A method that tries all possible keys until the correct decryption is found.

Encryption: The process of converting readable data into unreadable form.

Decryption: The reverse process of encryption.

ASCII: A numerical representation of characters used in computing.

9 Project Steps

1. Generate a random message using Python
2. Encrypt message using Caesar cipher
3. Encrypt message using Labyrinthe cipher
4. Measure execution time
5. Perform brute-force attack on Caesar cipher
6. Perform brute-force attack on Labyrinthe cipher
7. Record execution times
8. Plot results using Matplotlib
9. Save graph as image
10. Insert graph into report

10 Conclusion

This study demonstrates that encryption security strongly depends on key space size and algorithmic complexity.

Simple ciphers such as Caesar are easily broken, while more complex schemes significantly increase computational effort required for attacks.

However, both methods remain insecure compared to modern encryption standards such as AES .

11 Appendix: Functions Used

- `random.choice()`: selects random characters
- `list.append()`: efficiently builds strings
- `"".join()`: concatenates list into string
- `ord()/chr()`: converts characters to ASCII values and back
- `time.perf_counter()`: measures execution time precisely
- `matplotlib`: generates graphs

This project was developed for educational purposes to understand the principles of encryption, brute-force attacks, and computational complexity.

MOUKA ILYASS